Identifiers and Variables

Lecture 4

Based on Slides of Dr. Norazah Yusof

Identifiers

- All the Java components —classes, variables, and methods—need names.
- In Java these names are called *identifiers*, and, there are **rules** for what constitutes a legal Java identifier.
- Beyond what's *legal*, Java programmers (and Sun) have created ***conventions*** for naming methods, variables, and classes.

Rules for naming Identifiers

- Identifiers must start with a letter, a currency character ($), or a connecting character such as the underscore (_). Identifiers cannot start with a number!
- After the first character, identifiers can contain any combination of letters, currency characters, connecting characters, or numbers.
- Identifier can be any length. In practice, there is no limit to the number of characters an identifier can contain.
- You can't use a Java keyword as an identifier. (See Appendix A, “Java Keywords,” for a list of reserved words).
- Identifiers in Java are case-sensitive; **`utm`** and **`Utm`** are two different identifiers.
- Identifier cannot be true, false or null.

Sun's Java Code Conventions

The naming standards that Sun recommends:

- **Classes and interfaces** The first letter should be capitalized, and if several words are linked together to form the name, the first letter of the inner words should be uppercase (a format that's sometimes called "`camelCase`").
  - For classes, the names should typically be nouns. For example:
    - Animal
    - Account
    - PrintWriter
  - For interfaces, the names should typically be adjectives like
    - Runnable
    - Serializable

Sun's Java Code Conventions

- **Methods** The first letter should be lowercase, and then normal `camelCase` rules should be used.
  - In addition, the names should typically be verb-noun pairs. For example:
    - getBalance
    - doCalculation
    - setCustomerName
    - calcArea

Sun's Java Code Conventions

- **Variables** Like methods, the `camelCase` format should be used, starting with a lowercase letter.
- Sun recommends short, meaningful names, which sounds good to us. Some examples:
  - `nama`
  - `buttonWidth`
  - `accountBalance`
  - `myString`

Sun's Java Code Conventions

- **Constants** Java constants are created by marking variables `static` and `final`.
- They should be named using uppercase letters with underscore characters as separators:
  - `MIN__HEIGHT`

Programming Style and Documentation

- Appropriate Comments
- Naming Conventions
- Proper Indentation and Spacing Lines
- Block Styles

Appropriate Comments

Include a summary at the beginning of the program to explain what the program does, its key features, its supporting data structures, and any unique techniques it uses.

Include your name, class section, instructor, date, and a brief description at the beginning of the program.

Proper Indentation and Spacing

- Indentation
  - Indent two spaces.
- Spacing
  - Use blank line to separate segments of the code.

Block Styles

Use end-of-line style for braces.

Next-line style

```
public class Test
{
  public static void main(String[] args)
  {
    System.out.println("Block Styles");
  }
}
```

End-of-line style

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Block Styles");
  }
}
```

Variables

- Variables are used to store data in a program.
- There are two types of variables in Java:
  1. **Primitives**
  2. **Reference variables**

Primitives Variables

1. A primitive can be one of eight types: char, boolean, byte, short, int, long, double, or float.
2. Once a primitive has been declared, its primitive type can never change, although in most cases its value can change.
3. Primitive variables can be declared as **class variables (statics)**, **instance variables**, **method parameters**, or **local variables**.
4. One or more primitives, of the same primitive type, can be declared in a single line.
5. Examples of primitive variable declarations:

   i. `char huruf;`

   ii. `boolean myBooleanPrimitive;`

   iii. `int x, y, z;  // declare three int primitives`

   iv. `double area;`

Declaring primitive type

- Example of declaring Java variables with *primitive types*

```
int numberOfBeans;
double oneWeight, totalWeight;
```

- The declaration of a variable can be combined with its initialization via an assignment statement

```
int count = 0;
double distance = 55 * .5;
char grade = 'A';
```

- Note that some variables can be initialized and others can remain uninitialized in the same declaration

```
int initialCount = 50, finalCount;
```

Display 1.2 Primitive Types

TYPE NAME	KIND OF VALUE	MEMORY USED	SIZE RANGE
`boolean`	`true` or `false`	1 byte	not applicable
`char`	single character (Unicode)	2 bytes	all Unicode characters
`byte`	integer	1 byte	−128 to 127
`short`	integer	2 bytes	−32768 to 32767
`int`	integer	4 bytes	−2147483648 to 2147483647
`long`	integer	8 bytes	−9223372036854775808 to 9223372036854775807
`float`	floating-point number	4 bytes	$-3.40282347 \times 10^{+38}$ to $-1.40239846 \times 10^{-45}$
`double`	floating-point number	8 bytes	$\pm 1.76769313486231570 \times 10^{+308}$ to $\pm 4.94065645841246544 \times 10^{-324}$

Reference Variables

1. A reference variable is used to refer to (or access) an object.
2. A reference variable is declared to be of a specific type and that type can never be changed.
3. A reference variable can be used to refer to any object of the declared type, or of a *subtype* of the declared type (a compatible type).
4. Reference variables can be declared as **static variables**, **instance variables**, **method parameters**, or **local variables**.
5. One or more reference variables, of the same type, can be declared in a single line.
6. Examples of reference variable declarations:
   i. `Object o;`
   ii. `Animal myNewPetReferenceVariable;`
   iii. `String s1, s2, s3; //declare three String vars.`

Instance Variables

1. Instance variables are defined inside the class, but outside of any method, and are only initialized when the class is instantiated.
2. Instance variables are the fields that belong to each unique object.
3. For example, the following code defines fields (instance variables) for the radius of circle objects:

```
public class Circle extends Object  //Class header
{
    private double radius;
```

4. Each circle instance will know its own radius. In other words, each instance can have its own unique values for this field.
5. The term "field," "instance variable," "property," or "attribute," mean virtually the same thing.

Local Variables

1. Local variables are variables declared within a method.
2. That means the local variable starts its life inside the method, it's also destroyed when the method has completed.
3. Local variables are always on the stack, not the heap.
4. Although the value of the variable might be passed into, say, another method that then stores the value in an instance variable, the variable itself lives only within the scope of the method.
5. Before a local variable can be *used*, it **must be *initialized*** with a value. For example:

```
class TestServer {
     public void logIn() {
       int count = 10;
     }
    }
```

Local Variables

1. Local variable declarations can't use most of the modifiers that can be applied to instance variables, such as **public** (or the other access modifiers **private, protected**), **transient**, **volatile**, **abstract**, or **static**.
2. But local variables can be marked **final**.

Local Variables

- A local variable can't be referenced in any code **outside the method** in which it's declared.
- For example,

```
class TestServer {
  public void logIn() {
    int count = 10;
  }
  public void doSomething(int i) {
   count = i; // Won't compile! Can't access
              // count outside method login()
  }
}
```

- It would be impossible to refer to the variable `count` anywhere else in the class except within the scope of the method `logIn()`.
- However, the value of `count` can be passed out of the method to take on a new life.

Local Variables

- It is possible to declare a local variable with the same name as an instance variable. It's known as *shadowing*, as the following code demonstrates:

```
public class TestServer {
  int count = 9; //An instance variable named count
  public void logIn(){
    int count = 10; // A local variable named count
    System.out.println("local variable count is "
       + count);
  }
  public void count () {
    System.out.println("instance variable count is "
    + count);
  }
  public static void main(String[] args) {
    new TestServer().logIn();
    new TestServer().count();
  }
 }
```

Local Variables

- The code produces the following output:

```
local variable count is 10
instance variable count is 9
```

Final Variables

1. Declaring a variable with the `final` keyword makes it a **constant**.
2. **Constant** represent permanent data that never changes.
3. For primitives, this means that once the variable is assigned a value, the value can't be altered.
4. Syntax to declare a constant:

```
final datatype CONSTANTNAME = VALUE;
```

5. Example:

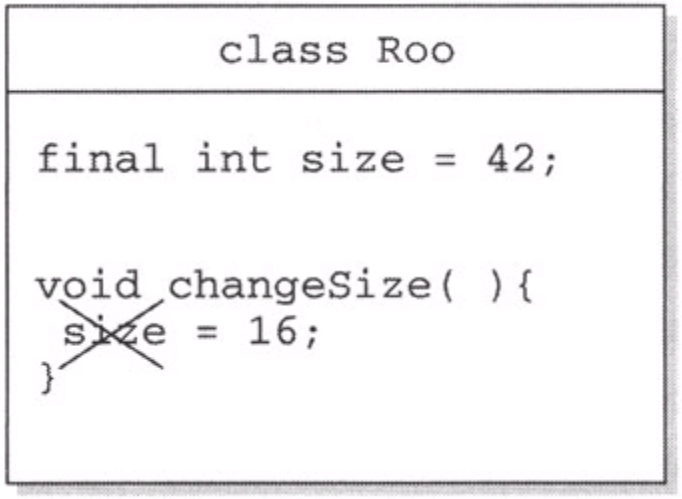
```
final double PI = 3.14159; //PI as constant
 final int SIZE = 3;
```

Final Variables (Is it legal?)

```
class Roo {
  final int size = 42;

  public void changeSize() {
    size = 16;
  }
}
```

final variable



final *variable* cannot be assigned a new value, once the initial method is made (the initial assignment of a value must happen before the constructor completes).

Final Variables

1. A reference variable marked `final` can't ever be reassigned to refer to a different object.
2. The data within the object can be modified, but the reference variable cannot be changed.
3. In other words, a final reference still allows you to modify the state of the object it refers to, but you can't modify the reference variable to make it refer to a different object.

Variable scoping

- Once a variable is declared and initialized, a natural question is "How long will this variable be around?"

```
public class Layout { // class
  static int s = 343; // static variable
  int x; // instance variable
  {  x = 7;
     int x2 = 5; } // initialization block

  public Layout() { // constructor
    x += 8;
    int x3 = 6;
  }

  public void doStuff() { // method
    int y = 0; // local variable
    for(int z = 0; z < 4; z++) { // 'for' code block
      y += z + x;
    }
  }
}
```

Variable scoping

- s is a static variable.
- x is an instance variable.
- y is a local variable (sometimes called a "method local" variable).
- z is a block variable.
- x2 is an init block variable, a flavor of local variable.
- x3 is a constructor variable, a flavor of local variable.

Variable scoping

- There are four basic scopes:
  1. Static variables have the longest scope; they are created when the class is loaded, and they survive as long as the class stays loaded in the JVM.
  2. Instance variables are the next most long-lived; they are created when a new instance is created, and they live until the instance is removed.
  3. Local variables are next; they live as long as their method remains on the stack. As we'll soon see, however, local variables can be alive, and still be "out of scope".
  4. Block variables live only as long as the code block is executing.

Scoping errors

- One common mistake happens when a variable is *shadowed* and two scopes overlap.
  1. Attempting to access an instance variable from a static context (typically from main()).
  2. Attempting to access a local variable from a nested method.
  3. Attempting to use a block variable after the code block has completed

Scoping errors

- One common mistake happens when a variable is *shadowed* and two scopes overlap.
  1. Attempting to access an instance variable from a static context (typically from main()).

     Example:

```
class ScopeErrors1 {
  int x = 5;
  public static void main(String[] args) {
   x++;   // won't compile, x is an 'instance' variable
  }
}
```

Scoping errors

2. Attempting to access a local variable from a nested method.

   Example:

```
public class ScopeErrors2 {
   public static void main(String [] args) {
     ScopeErrors s = new ScopeErrors();
     s.go() ;  }

   void go() {
     int y = 5;
     go2() ;
     y++;   // once go2() completes, y is back in scope  }

  void go2() {
     y++;          // won't compile, y is local to go()
  }
}
```

Scoping errors

3. Attempting to use a block variable after the code block has completed.

   Example:

```
void go3() {
 for(int z = 0; z < 5; z++) {
boolean test = false;
 if(z == 3) {      test = true;      break;
}
}  System.out.print(test);   // 'test' is an ex-variable,
                              // it has ceased to be...}
```